EV CHARGING STATION BOOKING PLATFORM

Complete Project Documentation

A peer-to-peer platform connecting EV owners with nearby household charger providers and public charging stations.

Backend	Spring Boot REST API
Database	PostgreSQL on Amazon RDS
Frontend	React on Amazon S3 static hosting; HTTPS delivery recommended
Authentication	Amazon Cognito
Deployment	Amazon EC2 + RDS + S3 + Cognito

Database schema • ER diagram • API contract • Workflow • AWS architecture • Phase-wise AI prompts • Testing and completion checklist

Document purpose

This document is the implementation blueprint for a college prototype. It covers the product idea, relational schema, ER relationships, API endpoints, workflows, AWS deployment and copy-ready prompts for building the app in phases. It is a plan—not a claim that every listed feature is already implemented.

Starting point: Spring Boot connects to PostgreSQL and GET /api/vehicles returns an empty list when there are no vehicle rows. Next: finish and test vehicle CRUD, then implement charging stations.

Contents

• 1. Project overview and scope

• 2. Roles and end-to-end workflow

• 3. Architecture and four AWS services

• 4. Relational database schema

• 5. ER diagram

• 6. REST API endpoint inventory

• 7. Implementation plan from start to finish

• 8. Phase-wise AI prompts

• 9. Testing, security and budget controls

• 10. Definition of done and submission checklist

1. Project overview

Problem statement

Public EV charging stations may be sparse or far apart. A driver whose battery is nearly depleted may need a nearby alternative. Individuals with compatible household chargers may be willing to share them under defined conditions.

Proposed solution

Build a peer-to-peer EV charging discovery and booking platform. Seekers register vehicles, share their current location while searching, view nearby household and public charging listings on a map, check availability and request a slot. Providers list chargers, location, connector, power, price and availability, then manage booking requests.

MVP scope

• Cognito-backed registration, email verification and sign-in.

• User profile linked to Cognito subject (sub).

• Vehicle CRUD for the logged-in owner.

• Household and public/business charger listings with coordinates.

• Nearby search by radius and compatible filters.

• Availability windows, booking requests, confirmation and cancellation.

• Simulated charging-session lifecycle.

• Station images in S3; object keys in PostgreSQL.

• React frontend, map markers, error/loading/empty states.

• Deployment to EC2, RDS, S3 and Cognito with cost monitoring.

Out of scope for the first prototype

• Real charger hardware control or automatic metering.

• Payment gateway and payouts.

• Live telematics, route optimization or AI predictions.

• Enterprise organization administration unless required.

• High-availability/multi-AZ production architecture.

2. Roles and end-to-end workflow

Role	Responsibilities
Seeker	Registers/logs in, registers vehicle, searches nearby chargers, checks availability and requests/cancels bookings.
Household provider	Lists a home charger, sets access, price and availability, manages booking requests.
Business/public provider	Lists commercial chargers and manages listings/availability.
Optional admin	Reviews or deactivates invalid listings.

User journey

• 1. User signs up through Cognito and verifies email.

• 2. React obtains tokens and sends a bearer token to Spring Boot.

• 3. Backend validates the token and creates/loads the app profile using unique Cognito sub.

• 4. Seeker registers a vehicle; ownership is derived from the authenticated user.

• 5. Provider lists a charger with type, access policy, coordinates, connector, power, price and availability.

• 6. Seeker grants device location permission; frontend sends latitude, longitude and radius.

• 7. Backend returns active, discoverable nearby stations; frontend plots user and charger markers.

• 8. Seeker selects a compatible charger/time and requests a booking.

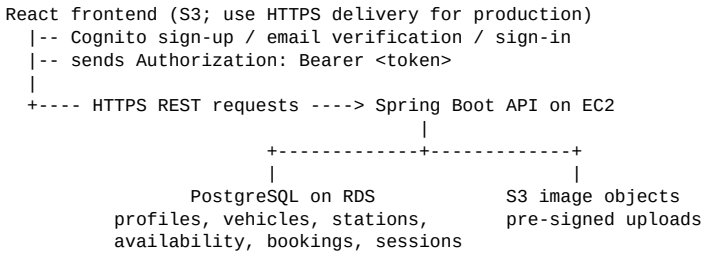
• 9. Backend checks ownership, access, compatibility, availability and overlapping reservations transactionally.

• 10. Provider confirms/rejects; seeker sees booking status.

• 11. A confirmed booking can start a simulated charging session and later be completed.

• 12. Station photos upload to S3 using pre-signed URLs; PostgreSQL stores the object key.

3. Architecture and AWS services



Service	Purpose	Implementation notes
Amazon EC2	Runs Spring Boot API	Expose only required ports; use HTTPS endpoint/reverse proxy; do not expose DB publicly.
Amazon RDS PostgreSQL	Managed relational database	Allow DB ingress only from backend security group; secure credentials and backups.
Amazon S3	React build and station image storage	S3 website endpoint is HTTP-only; use CloudFront or another HTTPS delivery method for production. Keep image bucket private where possible.
Amazon Cognito	Registration, email verification, sign-in and tokens	Backend verifies JWT signature, issuer, expiry and token use; link profile by Cognito sub.

Budget caution: service costs and credit eligibility depend on account offer, region, configuration and current pricing. A $20 target is not guaranteed by architecture alone. Set a budget alert, verify prices before provisioning, use minimal resources, and delete resources after demos.

Configuration

Use a configurable API base URL (e.g. VITE_API_BASE_URL), not hardcoded localhost. Set Spring CORS to the exact frontend origin. S3 static website hosting alone is HTTP-only; use an HTTPS delivery layer such as CloudFront for a production-style site.

4. Relational database schema

Seven core tables are recommended for the MVP. Use generated IDs, foreign keys, validation constraints, indexes and consistent timestamps (TIMESTAMPTZ recommended).

4.1 users

Column	Type / constraint	Meaning
id	BIGINT PK GENERATED	Internal app user ID
cognito_sub	VARCHAR(128) UNIQUE NOT NULL	Cognito subject
full_name	VARCHAR(120) NOT NULL	Display name
email	VARCHAR(254) UNIQUE NOT NULL	Profile email
phone	VARCHAR(30) NULL	Optional contact
created_at	TIMESTAMPTZ	Creation time
updated_at	TIMESTAMPTZ	Update time

4.2 vehicles

Column	Type / constraint	Meaning
id	BIGINT PK GENERATED	Vehicle ID
owner_user_id	BIGINT FK users.id NOT NULL	Registered owner
vehicle_type	VARCHAR(20) NOT NULL	BIKE, SCOOTER, CAR, OTHER
brand	VARCHAR(80) NULL	Manufacturer
model	VARCHAR(80) NOT NULL	Model
registration_number	VARCHAR(30) NULL	Optional; consider uniqueness
battery_capacity_kwh	NUMERIC(7,2) NULL CHECK > 0	Battery capacity
connector_type	VARCHAR(40) NULL	Charging type
created_at	TIMESTAMPTZ	Creation time

4.3 charging_stations

Column	Type / constraint	Meaning
id	BIGINT PK GENERATED	Station/listing ID
owner_user_id	BIGINT FK users.id NULL	Provider; null only for platform-managed listings
station_name	VARCHAR(140) NOT NULL	Display name
provider_type	VARCHAR(20) NOT NULL	HOUSEHOLD, BUSINESS, GOVERNMENT
access_type	VARCHAR(20) NOT NULL	PUBLIC or PRIVATE_LISTING/access policy
address	TEXT NOT NULL	Human-readable location
latitude	NUMERIC(9,6) CHECK -90..90	Latitude
longitude	NUMERIC(9,6) CHECK -180..180	Longitude
connector_type	VARCHAR(40) NOT NULL	Connector
power_kw	NUMERIC(7,2) CHECK > 0	Power
price_per_kwh	NUMERIC(8,2) NULL CHECK >= 0	Price
is_active	BOOLEAN DEFAULT TRUE	Discoverability
created_at	TIMESTAMPTZ	Creation time

4.4 bookings

Column	Type / constraint	Meaning
id	BIGINT PK GENERATED	Booking ID
seeker_user_id	BIGINT FK users.id NOT NULL	Authenticated seeker
vehicle_id	BIGINT FK vehicles.id NOT NULL	Vehicle to charge
station_id	BIGINT FK charging_stations.id NOT NULL	Selected listing
start_time	TIMESTAMPTZ NOT NULL	Requested start
end_time	TIMESTAMPTZ NOT NULL; end > start	Requested end
status	VARCHAR(20) NOT NULL	PENDING, CONFIRMED, REJECTED, CANCELLED, COMPLETED
estimated_cost	NUMERIC(10,2) NULL CHECK >= 0	Estimate at booking time
created_at	TIMESTAMPTZ	Request time

4.5 charging_sessions

Column	Type / constraint	Meaning
id	BIGINT PK GENERATED	Session ID
booking_id	BIGINT UNIQUE FK bookings.id	At most one session per booking
actual_start_time	TIMESTAMPTZ NULL	Actual/simulated start
actual_end_time	TIMESTAMPTZ NULL	End time
energy_consumed_kwh	NUMERIC(9,3) NULL CHECK >= 0	Measured/simulated energy
final_cost	NUMERIC(10,2) NULL CHECK >= 0	Final/simulated cost
status	VARCHAR(20) NOT NULL	IN_PROGRESS, COMPLETED, INTERRUPTED

4.6 charger_availability

Column	Type / constraint	Meaning
id	BIGINT PK GENERATED	Window ID
station_id	BIGINT FK charging_stations.id	Listing
start_time	TIMESTAMPTZ NOT NULL	Window start
end_time	TIMESTAMPTZ NOT NULL; end > start	Window end
status	VARCHAR(20) NOT NULL	AVAILABLE, UNAVAILABLE

4.7 station_images

Column	Type / constraint	Meaning
id	BIGINT PK GENERATED	Image metadata ID
station_id	BIGINT FK charging_stations.id	Listing
s3_object_key	VARCHAR(512) UNIQUE NOT NULL	S3 object key, not file bytes
created_at	TIMESTAMPTZ	Upload time

Integrity rules and indexes

• Index vehicles(owner_user_id), charging_stations(owner_user_id), bookings(seeker_user_id), bookings(station_id, start_time, end_time), charger_availability(station_id, start_time), station_images(station_id).

• For MVP nearby searches, use a correct PostgreSQL distance query; consider PostGIS only if needed later.

• Prevent double-booking with transaction-safe checks and preferably database-level protection; application-only checks can race.

• Verify vehicle ownership before booking. Use soft deactivation for stations with history.

• Validate latitude/longitude, positive power, non-negative price and end time after start.

• Never store Cognito passwords in PostgreSQL; never trust client-supplied owner IDs for authorization.

5. ER diagram

Paste this Mermaid ER diagram into a Mermaid-compatible editor or Markdown renderer.

```
erDiagram
    USERS ||--o{ VEHICLES : owns
    USERS o|--o{ CHARGING_STATIONS : provides
    USERS ||--o{ BOOKINGS : requests
    VEHICLES ||--o{ BOOKINGS : used_for
    CHARGING_STATIONS ||--o{ BOOKINGS : booked_at
    CHARGING_STATIONS ||--o{ CHARGER_AVAILABILITY : has
    BOOKINGS ||--o| CHARGING_SESSIONS : creates
    CHARGING_STATIONS ||--o{ STATION_IMAGES : has

    USERS {
      bigint id PK
      varchar cognito_sub UK
      varchar full_name
      varchar email UK
      varchar phone
    }
    VEHICLES {
      bigint id PK
      bigint owner_user_id FK
      varchar vehicle_type
      varchar brand
      varchar model
      decimal battery_capacity_kwh
      varchar connector_type
    }
    CHARGING_STATIONS {
      bigint id PK
      bigint owner_user_id FK
      varchar station_name
      varchar provider_type
      varchar access_type
      decimal latitude
      decimal longitude
      varchar connector_type
      decimal power_kw
      decimal price_per_kwh
      boolean is_active
    }
    BOOKINGS {
      bigint id PK
      bigint seeker_user_id FK
      bigint vehicle_id FK
      bigint station_id FK
      timestamp start_time
      timestamp end_time
      varchar status
    }
    CHARGING_SESSIONS {
      bigint id PK
      bigint booking_id FK
      timestamp actual_start_time
      timestamp actual_end_time
      decimal energy_consumed_kwh
      decimal final_cost
      varchar status
    }
    CHARGER_AVAILABILITY {
      bigint id PK
      bigint station_id FK
      timestamp start_time
      timestamp end_time
      varchar status
    }
    STATION_IMAGES {
      bigint id PK
      bigint station_id FK
```

```
    varchar s3_object_key
}
```

One user may own many vehicles and list many chargers. A booking belongs to one seeker, one vehicle and one station. A booking may create zero or one session. A station has many availability windows and images.

6. REST API endpoint inventory

Base path: /api. These are planned endpoints, not a claim that they are all implemented. User-specific routes require Cognito JWT validation and authorization.

6.1 User profiles

Method	Endpoint	Purpose
POST	/users/me	Create/sync profile after authentication
GET	/users/me	Read current profile
PUT	/users/me	Update profile
GET	/users/me/vehicles	List current user's vehicles
GET	/users/me/stations	List current user's listings

6.2 Vehicles

Method	Endpoint	Purpose
POST	/vehicles	Create vehicle owned by current user
GET	/vehicles	List current user's vehicles
GET	/vehicles/{vehicleId}	Read authorized vehicle
PUT	/vehicles/{vehicleId}	Update authorized vehicle
DELETE	/vehicles/{vehicleId}	Delete if business rules permit

6.3 Charging stations

Method	Endpoint	Purpose
POST	/stations	Create charger listing
GET	/stations	Browse active discoverable stations
GET	/stations/{stationId}	Read station details
PUT	/stations/{stationId}	Update owned listing
DELETE	/stations/{stationId}	Deactivate listing
GET	/stations/nearby?latitude={lat}&longitude;={lng}&radiusKm;={km}	Search within radius
GET	/stations/{stationId}/availability?date={yyyy-MM-dd}	Read availability
GET	/stations/mine	List provider's listings

6.4 Bookings

Method	Endpoint	Purpose
POST	/bookings	Request a slot
GET	/bookings	List current user's bookings
GET	/bookings/{bookingId}	Read authorized booking
PUT	/bookings/{bookingId}/cancel	Cancel if permitted
PUT	/bookings/{bookingId}/confirm	Provider confirms pending request
PUT	/bookings/{bookingId}/reject	Provider rejects pending request
GET	/stations/{stationId}/bookings	Provider views station bookings

6.5 Charging sessions

Method	Endpoint	Purpose
POST	/bookings/{bookingId}/sessions	Start eligible session
GET	/sessions/{sessionId}	Read authorized session
PUT	/sessions/{sessionId}/complete	Complete session with energy/cost
GET	/users/me/sessions	Current user's history

6.6 Availability

Method	Endpoint	Purpose
POST	/stations/{stationId}/availability	Create availability window
GET	/stations/{stationId}/availability	List windows
PUT	/stations/{stationId}/availability/{availabilityId}	Update window
DELETE	/stations/{stationId}/availability/{availabilityId}	Delete if no conflicting booking

6.7 S3 station images

Method	Endpoint	Purpose
POST	/stations/{stationId}/images/upload-url	Generate short-lived pre-signed upload URL
GET	/stations/{stationId}/images	List image metadata
DELETE	/stations/{stationId}/images/{imageId}	Delete authorized image/object

API conventions

• 201 Created for creation; 200 for reads/updates; 204 where appropriate; 400 invalid input; 401 unauthenticated; 403 forbidden; 404 missing resource; 409 booking conflict.

• Use DTOs, validation and consistent error responses instead of exposing JPA entities directly.

• Derive current user from verified JWT. Do not accept arbitrary user IDs as proof of identity.

• Do not hard-delete records with important booking history; deactivate or reject deletion.

7. Implementation plan: start to completion

Phase 0 — Scope and repository baseline

Inspect repo/branch, README, versions, environment config and team responsibilities. Agree API contract.

Done:project builds, secrets are absent from Git and the contract is shared.

Phase 1 — Stabilize backend

Verify DB connection and current GET /api/vehicles. Clean packages/config and add consistent errors.

Done:backend starts reliably and baseline tests pass.

Phase 2 — Vehicle CRUD

Implement POST, GET by ID, PUT and DELETE with validation and persistence tests.

Done:CRUD and negative tests pass.

Phase 3 — Charging listings

Implement entity/repository/service/controller, types, access policy, coordinates, connector, power, price and listing CRUD.

Done:providers can create/update/deactivate persistent listings.

Phase 4 — Nearby search

Implement radius query, coordinate validation and nearest-first results; share response contract with frontend teammate.

Done:nearby records are included and far/inactive records excluded.

Phase 5 — Cognito and profiles

Configure user pool/email verification, Spring Security JWT validation and profile sync by cognito_sub.

Done:valid tokens work and invalid/missing tokens are rejected.

Phase 6 — Availability and bookings

Implement time windows, booking state transitions, ownership, transactions and overlap protection.

Done:conflicting reservations cannot both be confirmed.

Phase 7 — Sessions and images

Add simulated session lifecycle and S3 pre-signed uploads/image metadata.

Done:session and image flows work with authorization.

Phase 8 — React frontend

Build auth, vehicle, provider listing, map/search, availability and booking screens with loading/error states.

Done:key journeys work against local API.

Phase 9 — Integration/security QA

Test tokens, CORS, ownership, invalid inputs, conflicts, network failures and end-to-end journeys.

Done:test results are recorded and critical defects fixed.

Phase 10 — AWS deployment

Configure EC2, RDS, S3, Cognito, HTTPS, secrets, security groups, health checks and budget monitoring.

Done:deployed app works without public DB exposure and budget is monitored.

Phase 11 — Demo/submission

Prepare README, diagrams, API docs, test evidence, cost notes, known limitations and demo script.

Done:clean-clone setup and 5-minute demo are reproducible.

8. Phase-wise prompts for AI-assisted implementation

Use one prompt at a time in an AI coding tool that can inspect your repository. Review the diff, run tests, commit a working milestone and only then continue. Do not ask an AI to generate the entire project in one pass.

Shared instruction — prepend to every prompt

```
You are a senior full-stack engineer helping build a college MVP: a peer-to-peer EV charging
discovery and booking platform. Existing backend: Spring Boot + Maven + PostgreSQL. Frontend:
React. AWS services selected: EC2, RDS PostgreSQL, S3 and Cognito.

Rules:
- Inspect repository, branch, README, build files and code before editing.
- Preserve working code, package names and versions unless explaining a necessary change.
- Implement only the current phase.
- List files changed and explain important decisions.
- Add validation, authorization, error handling and tests appropriate to the phase.
- Never commit credentials, AWS keys, DB passwords or real user data.
- Simulated charging measurements are allowed only because the MVP has no hardware integration.
- Run tests/build and report exact results. Never claim a command passed if it was not run.
- Update docs, provide manual verification steps and suggest a Git commit message.
```

Prompt 1 — Audit and stabilize current backend

```
Inspect the repo and identify the Spring Boot application, Vehicle entity/repository/service/controller,
PostgreSQL configuration and GET /api/vehicles. Do not rewrite working code. Report packages,
dependencies and Java/Spring versions. Check that secrets are environment variables and not tracked.
Run Maven tests/build if possible. Make only essential fixes to achieve a clean baseline. Document
how to test the current endpoint.
```

Prompt 2 — Vehicle CRUD

```
Implement POST /api/vehicles, GET /api/vehicles, GET /api/vehicles/{id},
PUT /api/vehicles/{id} and DELETE /api/vehicles/{id}. Use DTOs, validation, constructor injection,
appropriate HTTP status codes and consistent errors. Keep repository/service/controller roles separate.
Add tests for CRUD, invalid fields, missing IDs and persistence. Do not use an arbitrary owner ID as
authorization. Run tests and document Postman/curl examples.
```

Prompt 3 — Charging station listings

```
Add ChargingStation entity/repository/service/controller. Fields: id, ownerUserId (nullable only for
platform-managed listings), stationName, providerType (HOUSEHOLD/BUSINESS/GOVERNMENT), accessType,
address, latitude, longitude, connectorType, powerKw, pricePerKwh, isActive, createdAt. Add CRUD:
POST /api/stations, GET /api/stations, GET /api/stations/{id}, PUT /api/stations/{id},
DELETE /api/stations/{id}, GET /api/stations/mine. Validate coordinate ranges, positive power and
non-negative price. Enforce provider ownership for edits. Add tests and update docs. Do not duplicate
existing classes or change frameworks without reason.
```

Prompt 4 — Nearby search

```
Implement GET /api/stations/nearby?latitude={lat}&longitude={lng}&radiusKm={km}. Validate inputs,
return active discoverable stations within radius sorted nearest first, and use a correct PostgreSQL
distance calculation. Include id, name, providerType, coordinates, address, connectorType, powerKw,
pricePerKwh and distanceKm. Test nearby, far-away, inactive and invalid-coordinate cases. Document
the response contract for React map markers.
```

Prompt 5 — Cognito and user profiles

```
Integrate Cognito User Pool authentication; Cognito handles signup, email verification and login.
Configure Spring Security OAuth2 resource server to validate JWT signature, issuer, expiry and token
use. Add user profile entity/repository/service with unique cognito_sub, fullName, email and optional
phone. Implement GET/POST/PUT /api/users/me using the authenticated subject. Never store passwords
or trust client-supplied IDs. Configure CORS via environment/configuration. Test missing/invalid
tokens and profile ownership. Document placeholders, never real secrets.
```

Prompt 6 — Ownership authorization

```
Ensure authenticated users can only list/read/update/delete their own vehicles. Assign vehicle owner
from the authenticated profile on create. Ensure provider-only station edits and seeker-only booking
actions are enforced. Add tests with two users to prove one cannot access the other's records.
Keep public station browsing separate from provider management.
```

Prompt 7 — Availability and bookings

```
Implement ChargerAvailability and Booking models, repositories, services, controllers and validation.
Endpoints: create/list/update/delete availability; POST /api/bookings; GET /api/bookings;
GET /api/bookings/{id}; PUT /api/bookings/{id}/cancel, /confirm and /reject;
GET /api/stations/{stationId}/bookings. Derive seeker from auth. Verify vehicle ownership, active
station, access, connector compatibility, valid future times and availability. Prevent overlapping
confirmed bookings with transaction-safe logic and, where practical, a PostgreSQL exclusion constraint.
Define state transitions, return 409 for conflict and test concurrency/state changes.
```

Prompt 8 — Simulated charging sessions

```
Implement a simulated ChargingSession linked one-to-one to a booking. Endpoints:
POST /api/bookings/{bookingId}/sessions, GET /api/sessions/{id},
PUT /api/sessions/{id}/complete, GET /api/users/me/sessions. Only eligible confirmed bookings can
start; prevent duplicate active sessions. Record times, optional simulated energy and cost, and status.
Clearly document that measurements are simulated without hardware. Add tests and docs.
```

Prompt 9 — S3 image uploads

```
Implement secure S3 pre-signed uploads and station_images metadata. Endpoints:
POST /api/stations/{id}/images/upload-url, GET /api/stations/{id}/images,
DELETE /api/stations/{id}/images/{imageId}. Only authorized providers can upload/delete. Validate
file size/type, generate non-guessable keys, keep bucket private, use short-lived URLs, and never put
AWS credentials in React. Document IAM permissions and env vars without secrets; add tests.
```

Prompt 10 — React frontend

```
Inspect existing frontend. Build responsive React screens for Cognito signup/sign-in/email verification,
profile, vehicle CRUD, provider charger CRUD, browser geolocation, nearby map/list, station details,
availability, booking request/list/cancel, provider confirm/reject, simulated session and station image
upload. Use configurable API base URL, shared API client attaching Cognito token, loading/empty/error
states and accessible form validation. Do not hardcode secrets or production localhost. Confirm map
provider API key/licensing/cost before use. Add tests where practical and document setup.
```

Prompt 11 — Integration/security audit

```
Review React, Spring Boot, PostgreSQL, Cognito and S3 end-to-end. Verify CORS, JWT validation,
ownership checks, API response contract, booking conflicts, session transitions, image permissions,
environment config and schema. Test success, invalid input, 401/403, 404 and 409 cases. Fix issues
without unnecessary architecture changes. Provide reproducible local setup and test plan. Do not
invent test results.
```

Prompt 12 — AWS deployment and cost control

```
Prepare deployment steps for EC2 Spring Boot, RDS PostgreSQL, S3 React assets/images and Cognito.
Use actual account region and current credit eligibility; do not assume free. Estimate costs with
assumptions and official pricing references. Include budget alerts, least-privilege IAM, private RDS,
security groups, secure secrets, HTTPS frontend delivery (S3 website endpoint alone is HTTP-only),
backend HTTPS, CORS, health checks, logs and teardown. Do not create resources or spend money
automatically. Use placeholders for secrets and include rollback/verification.
```

Prompt 13 — Final QA and submission

```
Run backend/frontend tests and builds, inspect Git diff and scan for secrets. List unresolved issues.
Produce README with versions, env vars, DB setup/migrations, Cognito, S3, local run, tests, API
inventory, deployment and teardown. Include ER/architecture diagrams, sample data, Postman collection
or curl examples, actual test results, budget checklist and a 5-minute seeker/provider demo script.
Do not invent screenshots, URLs, credentials or test results. Ensure reproducibility from clean clone.
```

9. Testing, security and budget controls

Area	Minimum evidence
Vehicles	Create/list/update/delete, invalid fields, missing ID, owner isolation.
Stations	Household/business listing, coordinate validation, inactive listing hidden, owner-only updates.
Nearby search	Nearby station included, far station excluded, invalid radius/coordinates rejected.
Bookings	Valid request, invalid time, unavailable slot, overlap conflict, cancel, confirm/reject, auth checks.
Cognito	Email verification, valid token, missing/invalid/expired token, profile mapped by sub.
S3 images	Authorized pre-signed upload, size/type checks, unauthorized deletion blocked.
Deployment	Production API URL, HTTPS, CORS, private RDS, health check, restart/recovery.

Budget checklist

• Check AWS billing/credits and current regional prices before provisioning; credits may exclude some charges or taxes.

• Create a budget alert; it is a notification, not a hard spending cap.

• Choose minimal eligible EC2/RDS resources only after price checks. Avoid NAT Gateway/load balancers and paid extras unless needed.

• Never open PostgreSQL to 0.0.0.0/0; restrict it to the backend security group.

• Email verification and SMS are different paths; avoid SMS features until pricing is understood.

• Stop/delete resources after demos. Stopping EC2 may leave EBS charges; RDS storage/snapshots may also incur costs.

• Never commit AWS keys, DB passwords, Cognito secrets or private pre-signed URLs.

10. Definition of done

• Clean clone builds using documented versions and steps.

• User can register/sign in with Cognito and create a profile.

• Seeker can register a vehicle and only access their own vehicles.

• Provider can create/manage a household/public charger listing.

• Seeker finds nearby active chargers and sees markers on a map.

• Seeker requests a slot; provider confirms/rejects; overlap conflicts are prevented.

• Confirmed booking can start/complete a clearly simulated session.

• Station images upload securely to S3 and appear in the UI.

• Backend tests/frontend build pass; limitations are documented.

• Deployment uses HTTPS, secure secrets, restricted RDS access and CORS.

• Team can explain schema, ER diagram, APIs, AWS services and budget.

• Resources can be cleaned up after evaluation.

Appendix A — Suggested repository structure

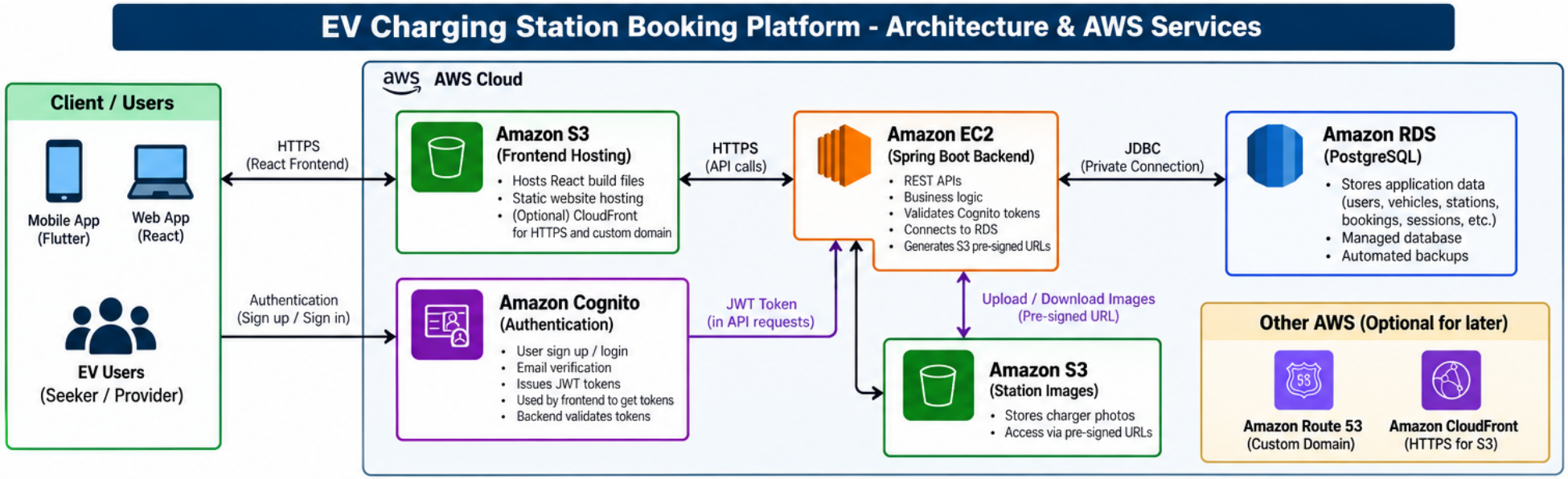
```
ev-charging-platform/
■■■ backend/
■   ■■■ src/main/java/.../
■   ■   ■■■ config/ controller/ dto/ entity/ exception/
■   ■   ■■■ repository/ security/ service/
■   ■■■ src/main/resources/
■   ■■■ pom.xml
■■■ frontend/
■   ■■■ src/api/ auth/ components/ features/ pages/ routes/
■   ■■■ package.json
■■■ docs/
■   ■■■ architecture.md database-schema.md api-contract.md
■   ■■■ testing.md deployment.md
■■■ postman/collection.json
■■■ .env.example
■■■ README.md
```

Appendix B — First next action

Commit the current working backend. Use Prompt 1 to audit the actual repository, then Prompt 2 to complete Vehicle CRUD. Review changes, run tests, and commit each working milestone before continuing to charging stations.

Appendix B — System Architecture & AWS Services

Core services: S3, EC2, RDS PostgreSQL and Cognito; CloudFront and Route 53 are optional additions.



Database ER Diagram (PostgreSQL - RDS)

users

PK	id	BIGINT
	cognito_sub	VARCHAR (unique)
	full_name	VARCHAR
	email	VARCHAR (unique)
	phone	VARCHAR
	created_at	TIMESTAMP

vehicles

PK	id	BIGINT
FK	owner_user_id	BIGINT
	vehicle_type	VARCHAR
	brand	VARCHAR
	model	VARCHAR
	registration_number	VARCHAR
	battery_capacity_kwh	DECIMAL
	connector_type	VARCHAR
	created_at	TIMESTAMP

charging_stations

PK	id	BIGINT
FK	owner_user_id	BIGINT (nullable)
	station_name	VARCHAR
	provider_type	VARCHAR
	access_type	VARCHAR
	address	TEXT
	latitude	DECIMAL(9,6)
	longitude	DECIMAL(9,6)
	connector_type	VARCHAR
	power_kw	DECIMAL
	price_per_kwh	DECIMAL
	is_active	BOOLEAN
	created_at	TIMESTAMP

bookings

PK	id	BIGINT
FK	seeker_user_id	BIGINT
FK	vehicle_id	BIGINT
FK	station_id	BIGINT
	start_time	TIMESTAMP
	end_time	TIMESTAMP
	status	VARCHAR
	estimated_cost	DECIMAL
	created_at	TIMESTAMP

charging_sessions

PK	id	BIGINT
FK	booking_id	BIGINT (unique)
	actual_start_time	TIMESTAMP
	actual_end_time	TIMESTAMP
	energy_consumed_kwh	DECIMAL
	final_cost	DECIMAL
	status	VARCHAR

charger_availability

PK	id	BIGINT
FK	station_id	BIGINT
	start_time	TIMESTAMP
	end_time	TIMESTAMP
	status	VARCHAR

station_images

PK	id	BIGINT
FK	station_id	BIGINT
	s3_object_key	VARCHAR
	created_at	TIMESTAMP

owns (users 1 — N vehicles)
used for (vehicles 1 — N bookings)
makes (users 1 — N bookings)
receives (bookings N — 1 charging_stations)
has (charging_stations 1 — N charger_availability)
displays (charging_stations 1 — N station_images)
generates (bookings 1 — 1 charging_sessions)

Legend
PK - Primary Key
FK - Foreign Key
1 - One (single record)
N - Many (multiple records)
— - Relationship

End-to-End Workflow

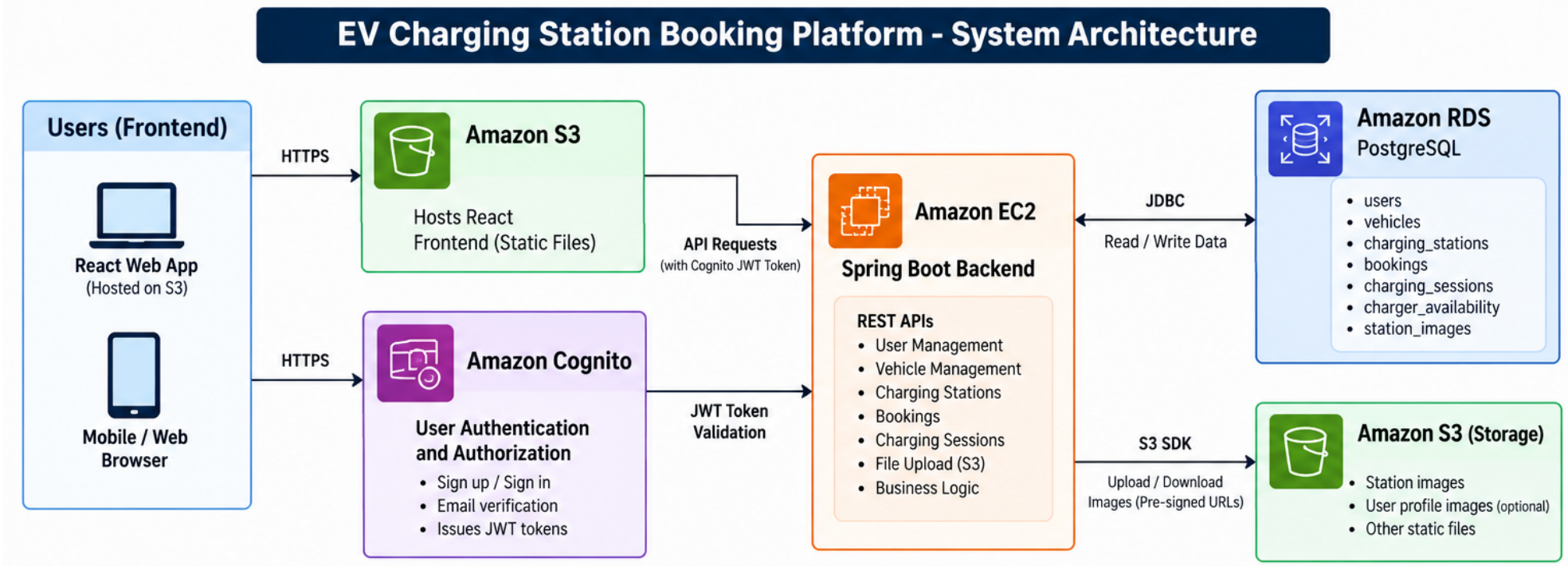
1. User signs up / logs in via **Amazon Cognito**
2. Frontend (React on S3) gets Cognito tokens
3. Frontend calls Spring Boot APIs on EC2 with JWT token
4. Backend validates token and processes request
5. Backend reads/writes data in Amazon RDS (PostgreSQL)
6. For station images, backend generates pre-signed S3 URL
7. Frontend uploads/downloads images directly from S3
8. Users can search nearby chargers, view details and book
9. Booking and session data are stored in **RDS**
10. Entire app is accessed over HTTPS (use CloudFront for S3 in production)

Key AWS Services Used

Service	Purpose
S3	Hosts React frontend and stores charger images (Static website hosting)
EC2	Runs Spring Boot backend (REST APIs, business logic)
RDS	PostgreSQL database for application data
Cognito	User authentication, sign up, login, email verification

Appendix C — System Architecture Overview

Request flow: client → frontend/authentication → Spring Boot APIs → database and image storage.



How the Architecture Works

1. User accesses the React app hosted on Amazon S3 using a browser.
2. User signs up / logs in using Amazon Cognito, which handles authentication and returns a JWT token.
3. Frontend calls Spring Boot APIs on EC2 with the JWT token.
4. Backend validates the token, applies business logic, and reads/writes data in Amazon RDS (PostgreSQL).
5. Station images are stored in Amazon S3 using pre-signed URLs.

End-to-End Flow

1. User Registration & Login
- Sign up / Sign in via Amazon Cognito
- Receive JWT token

2. Access Frontend
- Open React app (hosted on S3)
- User interacts with UI

3. Search Charging Stations
- Send location to backend
- Get nearby chargers from EC2 (RDS data)
- Display on map

4. Book a Charging Slot
- Select station and time slot
- Create booking via API
- Backend checks availability and stores in RDS

5. Charging Session
- Start / complete session
- Record charging details in RDS
- Update session status

6. View History & Manage
- View past bookings
- Manage vehicles and charging stations